

King Fahd University of Petroleum and Minerals

ICS-344 - Information Security

Section 1 & 2

DVSA Vulnerability Discovery and Remediation

Term: 252

Student Name(s): Husain Al Muallim, Ali Al Qatari

Student ID(s): 202163730, 202279780

DVSA Website URL: *https://your-dvsa-site.example*

AWS Region: us-east-1

Date: *Submission date*

Project Report

Contents

Project Overview	v
1 Lesson 01: Event Injection	1
1.1 Part 1: Goal and Vulnerability Summary	1
1.2 Part 2: Why This Works / Root Cause	1
1.3 Part 3: Environment and Setup	1
1.4 Part 4: Reproduction Steps	1
1.5 Part 5: Evidence and Proof	2
1.6 Part 6: Fix Strategy / Probable Mitigation	2
1.7 Part 7: Code / Config Changes	2
1.8 Part 8: Verification After Fix	3
1.9 Part 9: Structured Operation and Security Analysis	4
1.10 Part 10: Takeaway / Lessons Learned	4
2 Lesson 02: Broken Authentication	5
2.1 Part 1: Goal and Vulnerability Summary	5
2.2 Part 2: Why This Works / Root Cause	5
2.3 Part 3: Environment and Setup	5
2.4 Part 4: Reproduction Steps	5
2.5 Part 5: Evidence and Proof	6
2.6 Part 6: Fix Strategy / Probable Mitigation	6
2.7 Part 7: Code / Config Changes	6
2.8 Part 8: Verification After Fix	7
2.9 Part 9: Structured Operation and Security Analysis	8
2.10 Part 10: Takeaway / Lessons Learned	8

3	Lesson 03: Sensitive Information Disclosure	9
3.1	Part 1: Goal and Vulnerability Summary	9
3.2	Part 2: Why This Works / Root Cause	9
3.3	Part 3: Environment and Setup	9
3.4	Part 4: Reproduction Steps	9
3.5	Part 5: Evidence and Proof	10
3.6	Part 6: Fix Strategy / Probable Mitigation	10
3.7	Part 7: Code / Config Changes	10
3.8	Part 8: Verification After Fix	11
3.9	Part 9: Structured Operation and Security Analysis	12
3.10	Part 10: Takeaway / Lessons Learned	12
4	Lesson 04: Insecure Cloud Configuration	13
4.1	Part 1: Goal and Vulnerability Summary	13
4.2	Part 2: Why This Works / Root Cause	13
4.3	Part 3: Environment and Setup	13
4.4	Part 4: Reproduction Steps	13
4.5	Part 5: Evidence and Proof	14
4.6	Part 6: Fix Strategy / Probable Mitigation	14
4.7	Part 7: Code / Config Changes	14
4.8	Part 8: Verification After Fix	15
4.9	Part 9: Structured Operation and Security Analysis	16
4.10	Part 10: Takeaway / Lessons Learned	16
5	Lesson 05: Broken Access Control	17
5.1	Part 1: Goal and Vulnerability Summary	17
5.2	Part 2: Why This Works / Root Cause	17
5.3	Part 3: Environment and Setup	17
5.4	Part 4: Reproduction Steps	17
5.5	Part 5: Evidence and Proof	18
5.6	Part 6: Fix Strategy / Probable Mitigation	18
5.7	Part 7: Code / Config Changes	18
5.8	Part 8: Verification After Fix	19

5.9	Part 9: Structured Operation and Security Analysis	20
5.10	Part 10: Takeaway / Lessons Learned	20
6	Lesson 06: Denial of Service	21
6.1	Part 1: Goal and Vulnerability Summary	21
6.2	Part 2: Why This Works / Root Cause	21
6.3	Part 3: Environment and Setup	21
6.4	Part 4: Reproduction Steps	21
6.5	Part 5: Evidence and Proof	22
6.6	Part 6: Fix Strategy / Probable Mitigation	22
6.7	Part 7: Code / Config Changes	22
6.8	Part 8: Verification After Fix	23
6.9	Part 9: Structured Operation and Security Analysis	24
6.10	Part 10: Takeaway / Lessons Learned	24
7	Lesson 07: Over-Privileged Function	25
7.1	Part 1: Goal and Vulnerability Summary	25
7.2	Part 2: Why This Works / Root Cause	25
7.3	Part 3: Environment and Setup	25
7.4	Part 4: Reproduction Steps	25
7.5	Part 5: Evidence and Proof	26
7.6	Part 6: Fix Strategy / Probable Mitigation	26
7.7	Part 7: Code / Config Changes	26
7.8	Part 8: Verification After Fix	27
7.9	Part 9: Structured Operation and Security Analysis	28
7.10	Part 10: Takeaway / Lessons Learned	28
8	Lesson 08: Logic Vulnerabilities	29
8.1	Part 1: Goal and Vulnerability Summary	29
8.2	Part 2: Why This Works / Root Cause	29
8.3	Part 3: Environment and Setup	29
8.4	Part 4: Reproduction Steps	29
8.5	Part 5: Evidence and Proof	30
8.6	Part 6: Fix Strategy / Probable Mitigation	30

8.7	Part 7: Code / Config Changes	30
8.8	Part 8: Verification After Fix	31
8.9	Part 9: Structured Operation and Security Analysis	32
8.10	Part 10: Takeaway / Lessons Learned	32
9	Lesson 09: Vulnerable Dependencies	33
9.1	Part 1: Goal and Vulnerability Summary	33
9.2	Part 2: Why This Works / Root Cause	33
9.3	Part 3: Environment and Setup	33
9.4	Part 4: Reproduction Steps	33
9.5	Part 5: Evidence and Proof	34
9.6	Part 6: Fix Strategy / Probable Mitigation	34
9.7	Part 7: Code / Config Changes	34
9.8	Part 8: Verification After Fix	35
9.9	Part 9: Structured Operation and Security Analysis	36
9.10	Part 10: Takeaway / Lessons Learned	36
10	Lesson 10: Unhandled Exceptions	37
10.1	Part 1: Goal and Vulnerability Summary	37
10.2	Part 2: Why This Works / Root Cause	37
10.3	Part 3: Environment and Setup	37
10.4	Part 4: Reproduction Steps	37
10.5	Part 5: Evidence and Proof	38
10.6	Part 6: Fix Strategy / Probable Mitigation	38
10.7	Part 7: Code / Config Changes	38
10.8	Part 8: Verification After Fix	39
10.9	Part 9: Structured Operation and Security Analysis	40
10.10	Part 10: Takeaway / Lessons Learned	40
	Optional Bonus Findings	41
	Conclusion	42

Project Overview

Scope

Summarize the project scope: deployment in a non-production AWS account, the 10 official lessons, remediation, and verification.

Deployment Summary

- DVSA Website URL: *https://your-dvsa-site.example*
- AWS region: *us-east-1*
- CloudFormation stack: *Stack name*
- Main services used: *S3, API Gateway, Lambda, DynamoDB, SQS, Cognito, IAM, CloudWatch*

Architecture and Request Pipeline

Explain the intended high-level request flow from browser to API Gateway to Lambda to backend services and back.

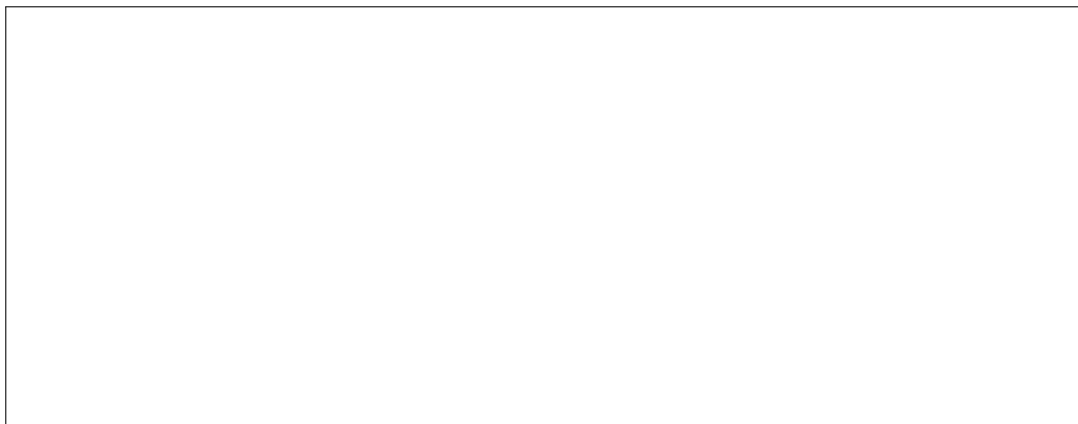


Figure 1: Optional architecture figure or request-flow diagram.

Methodology and Evidence Sources

- *How the environment was validated before testing.*
- *What tools were used to reproduce and verify vulnerabilities.*
- *How evidence was captured and redacted.*
- *How fixes were verified after remediation.*

Chapter 1

Lesson 01: Event Injection

1.1 Part 1: Goal and Vulnerability Summary

State the vulnerability type, main affected component, security impact, and the expected safe behavior for Event Injection.

1.2 Part 2: Why This Works / Root Cause

Explain the security mistake that makes this unsafe input handling issue possible. Keep this conceptual and focused on root cause rather than symptoms.

1.3 Part 3: Environment and Setup

- DVSA Website URL / API endpoint: *Fill in the exact endpoint or page used.*
- AWS services involved: *For example API Gateway, Lambda, Cognito, S3, DynamoDB, SQS, IAM.*
- Relevant function, role, or log group: *Name the exact resources used in this lesson.*
- User or test context: *Attacker, victim, admin, or anonymous context as applicable.*
- Tools used: *Browser DevTools, curl, jq, AWS Console, AWS CLI, Policy Simulator, CloudWatch, etc.*

1.4 Part 4: Reproduction Steps

1. *Describe the baseline or normal behavior first.*
2. *List the exact steps that trigger the vulnerability.*
3. *Include the request, timing, role, or payload details needed for reproducibility.*

4. *Record the observable result that proves the issue.*

1.5 Part 5: Evidence and Proof

- *Attach the strongest pre-fix evidence for this lesson.*
- *Use only evidence that directly proves the claim.*
- *Redact real tokens, secrets, and temporary credentials before submission.*



Figure 1.1: Replace with pre-fix evidence for Lesson 01 (Event Injection).

1.6 Part 6: Fix Strategy / Probable Mitigation

- **Fix location:** *Identify the backend, IAM, cloud configuration, or workflow layer that must change.*
- **Security property restored:** *For example authentication integrity, authorization, least privilege, or safe input handling.*
- **Why this addresses root cause:** *Explain why the fix is real and not cosmetic.*

1.7 Part 7: Code / Config Changes

Paste the real code diff, policy change, CloudFormation change, or configuration update below.

Replace this box with the actual diff, code snippet, policy JSON, or configuration change.

Figure 1.2: Replace with the concrete code or configuration change for Lesson 01.

1.8 Part 8: Verification After Fix

1. *Repeat the same test that proved the vulnerability.*
2. *Show that the exploit no longer succeeds.*
3. *Also show that normal intended behavior still works.*

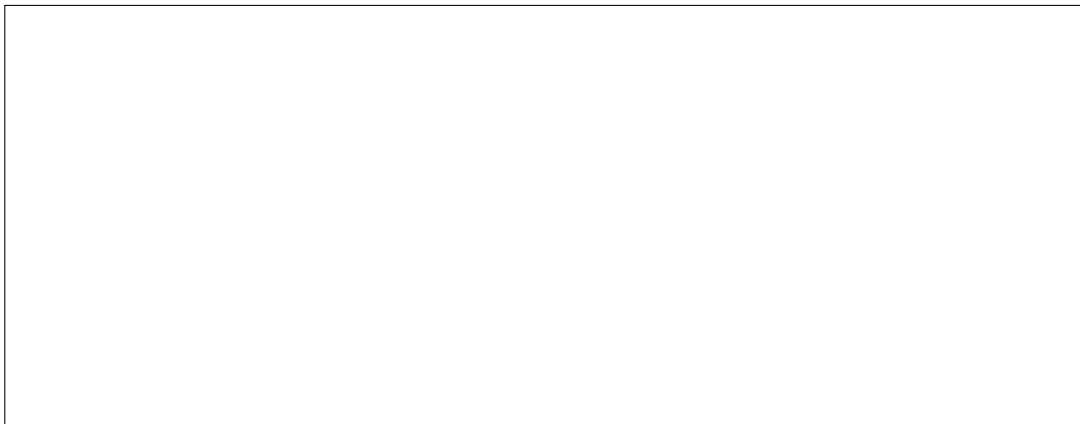


Figure 1.3: Replace with post-fix verification evidence for Lesson 01 (Event Injection).

1.9 Part 9: Structured Operation and Security Analysis

Table A

Intended rule(s)	Artifacts used to infer the rule	Normal evidence	Exploit evidence
<i>State the intended security or workflow rule.</i>	<i>Identify the source used to infer the rule: UI flow, code path, IAM policy, logs, helper guide, etc.</i>	<i>Show normal behavior when the rule is respected.</i>	<i>Show behavior proving the rule was violated.</i>

Table B

Why the exploit is a deviation	Deviation class	Fix applied	Post-fix verification	Optional latency / timing note
<i>Explain why the observed result violates the intended rule.</i>	<i>Examples: authentication failure, access control failure, unsafe input handling, logic flaw, DoS, excessive privilege.</i>	<i>Summarize the real corrective action.</i>	<i>Describe the evidence that the deviation is closed.</i>	<i>Use this only when timing or latency matters.</i>

1.10 Part 10: Takeaway / Lessons Learned

End with a short security lesson that ties the unsafe input handling weakness back to serverless design, AWS service interaction, and secure engineering practice.

Chapter 2

Lesson 02: Broken Authentication

2.1 Part 1: Goal and Vulnerability Summary

State the vulnerability type, main affected component, security impact, and the expected safe behavior for Broken Authentication.

2.2 Part 2: Why This Works / Root Cause

Explain the security mistake that makes this authentication failure issue possible. Keep this conceptual and focused on root cause rather than symptoms.

2.3 Part 3: Environment and Setup

- DVSA Website URL / API endpoint: *Fill in the exact endpoint or page used.*
- AWS services involved: *For example API Gateway, Lambda, Cognito, S3, DynamoDB, SQS, IAM.*
- Relevant function, role, or log group: *Name the exact resources used in this lesson.*
- User or test context: *Attacker, victim, admin, or anonymous context as applicable.*
- Tools used: *Browser DevTools, curl, jq, AWS Console, AWS CLI, Policy Simulator, CloudWatch, etc.*

2.4 Part 4: Reproduction Steps

1. *Describe the baseline or normal behavior first.*
2. *List the exact steps that trigger the vulnerability.*
3. *Include the request, timing, role, or payload details needed for reproducibility.*

4. *Record the observable result that proves the issue.*

2.5 Part 5: Evidence and Proof

- *Attach the strongest pre-fix evidence for this lesson.*
- *Use only evidence that directly proves the claim.*
- *Redact real tokens, secrets, and temporary credentials before submission.*

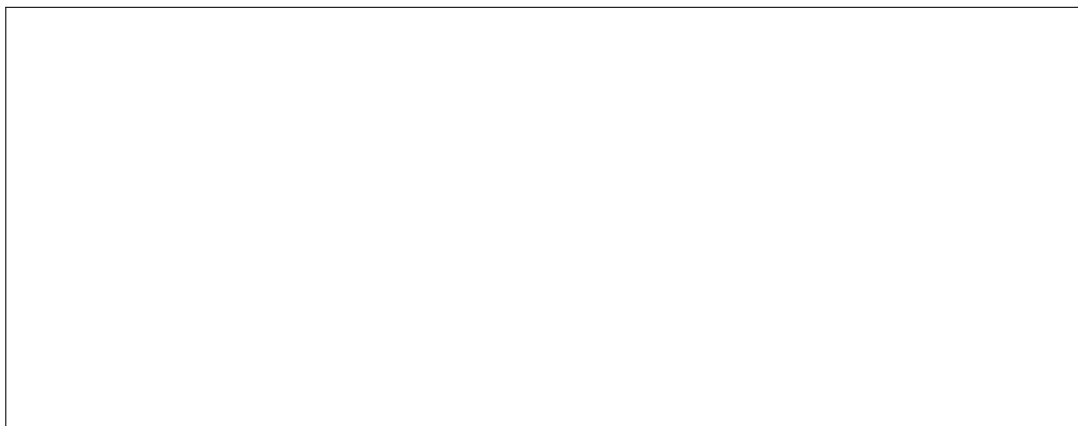


Figure 2.1: Replace with pre-fix evidence for Lesson 02 (Broken Authentication).

2.6 Part 6: Fix Strategy / Probable Mitigation

- **Fix location:** *Identify the backend, IAM, cloud configuration, or workflow layer that must change.*
- **Security property restored:** *For example authentication integrity, authorization, least privilege, or safe input handling.*
- **Why this addresses root cause:** *Explain why the fix is real and not cosmetic.*

2.7 Part 7: Code / Config Changes

Paste the real code diff, policy change, CloudFormation change, or configuration update below.

Replace this box with the actual diff, code snippet, policy JSON, or configuration change.

Figure 2.2: Replace with the concrete code or configuration change for Lesson 02.

2.8 Part 8: Verification After Fix

1. *Repeat the same test that proved the vulnerability.*
2. *Show that the exploit no longer succeeds.*
3. *Also show that normal intended behavior still works.*

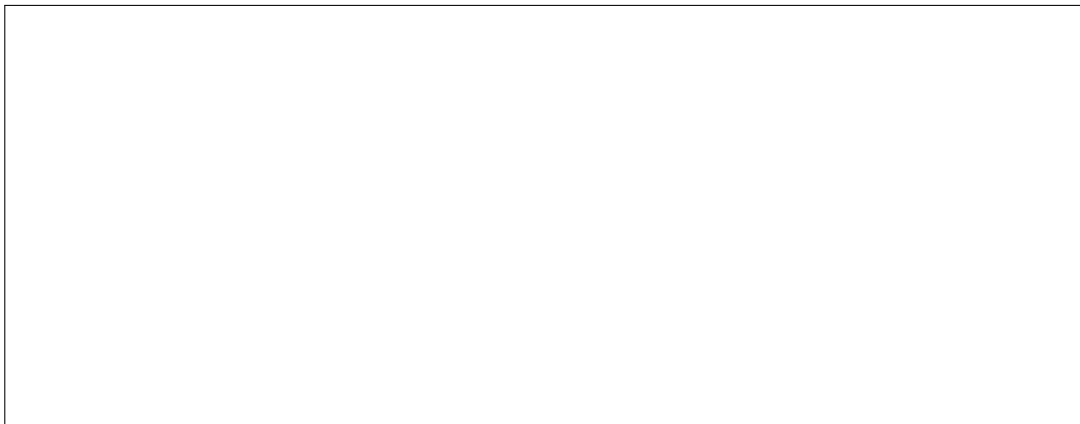


Figure 2.3: Replace with post-fix verification evidence for Lesson 02 (Broken Authentication).

2.9 Part 9: Structured Operation and Security Analysis

Table A

Intended rule(s)	Artifacts used to infer the rule	Normal evidence	Exploit evidence
<i>State the intended security or workflow rule.</i>	<i>Identify the source used to infer the rule: UI flow, code path, IAM policy, logs, helper guide, etc.</i>	<i>Show normal behavior when the rule is respected.</i>	<i>Show behavior proving the rule was violated.</i>

Table B

Why the exploit is a deviation	Deviation class	Fix applied	Post-fix verification	Optional latency / timing note
<i>Explain why the observed result violates the intended rule.</i>	<i>Examples: authentication failure, access control failure, unsafe input handling, logic flaw, DoS, excessive privilege.</i>	<i>Summarize the real corrective action.</i>	<i>Describe the evidence that the deviation is closed.</i>	<i>Use this only when timing or latency matters.</i>

2.10 Part 10: Takeaway / Lessons Learned

End with a short security lesson that ties the authentication failure weakness back to serverless design, AWS service interaction, and secure engineering practice.

Chapter 3

Lesson 03: Sensitive Information Disclosure

3.1 Part 1: Goal and Vulnerability Summary

State the vulnerability type, main affected component, security impact, and the expected safe behavior for Sensitive Information Disclosure.

3.2 Part 2: Why This Works / Root Cause

Explain the security mistake that makes this sensitive data exposure issue possible. Keep this conceptual and focused on root cause rather than symptoms.

3.3 Part 3: Environment and Setup

- DVSA Website URL / API endpoint: *Fill in the exact endpoint or page used.*
- AWS services involved: *For example API Gateway, Lambda, Cognito, S3, DynamoDB, SQS, IAM.*
- Relevant function, role, or log group: *Name the exact resources used in this lesson.*
- User or test context: *Attacker, victim, admin, or anonymous context as applicable.*
- Tools used: *Browser DevTools, curl, jq, AWS Console, AWS CLI, Policy Simulator, CloudWatch, etc.*

3.4 Part 4: Reproduction Steps

1. *Describe the baseline or normal behavior first.*
2. *List the exact steps that trigger the vulnerability.*

3. *Include the request, timing, role, or payload details needed for reproducibility.*
4. *Record the observable result that proves the issue.*

3.5 Part 5: Evidence and Proof

- *Attach the strongest pre-fix evidence for this lesson.*
- *Use only evidence that directly proves the claim.*
- *Redact real tokens, secrets, and temporary credentials before submission.*

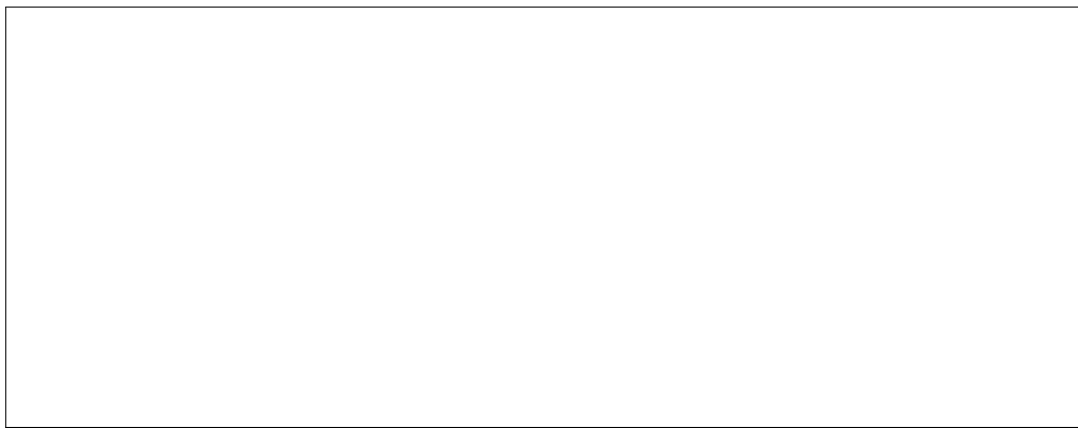


Figure 3.1: Replace with pre-fix evidence for Lesson 03 (Sensitive Information Disclosure).

3.6 Part 6: Fix Strategy / Probable Mitigation

- **Fix location:** *Identify the backend, IAM, cloud configuration, or workflow layer that must change.*
- **Security property restored:** *For example authentication integrity, authorization, least privilege, or safe input handling.*
- **Why this addresses root cause:** *Explain why the fix is real and not cosmetic.*

3.7 Part 7: Code / Config Changes

Paste the real code diff, policy change, CloudFormation change, or configuration update below.

Replace this box with the actual diff, code snippet, policy JSON, or configuration change.

Figure 3.2: Replace with the concrete code or configuration change for Lesson 03.

3.8 Part 8: Verification After Fix

1. *Repeat the same test that proved the vulnerability.*
2. *Show that the exploit no longer succeeds.*
3. *Also show that normal intended behavior still works.*

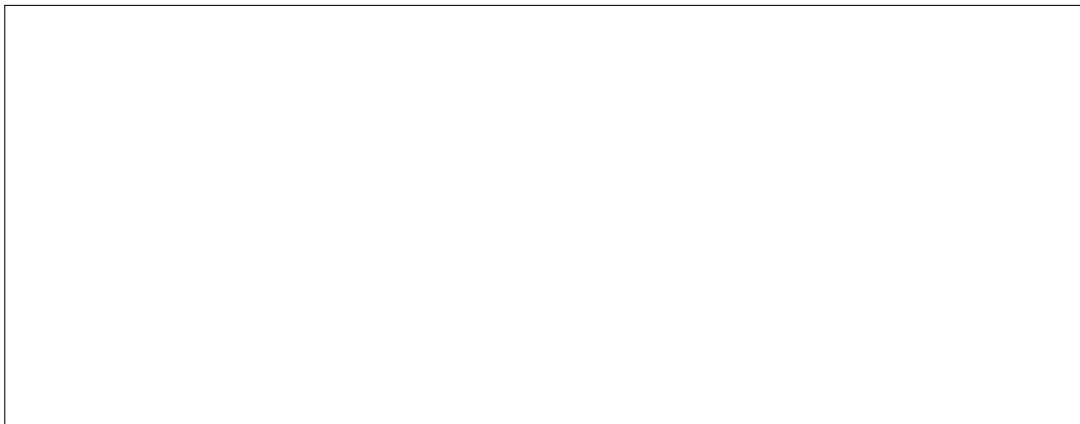


Figure 3.3: Replace with post-fix verification evidence for Lesson 03 (Sensitive Information Disclosure).

3.9 Part 9: Structured Operation and Security Analysis

Table A

Intended rule(s)	Artifacts used to infer the rule	Normal evidence	Exploit evidence
<i>State the intended security or workflow rule.</i>	<i>Identify the source used to infer the rule: UI flow, code path, IAM policy, logs, helper guide, etc.</i>	<i>Show normal behavior when the rule is respected.</i>	<i>Show behavior proving the rule was violated.</i>

Table B

Why the exploit is a deviation	Deviation class	Fix applied	Post-fix verification	Optional latency / timing note
<i>Explain why the observed result violates the intended rule.</i>	<i>Examples: authentication failure, access control failure, unsafe input handling, logic flaw, DoS, excessive privilege.</i>	<i>Summarize the real corrective action.</i>	<i>Describe the evidence that the deviation is closed.</i>	<i>Use this only when timing or latency matters.</i>

3.10 Part 10: Takeaway / Lessons Learned

End with a short security lesson that ties the sensitive data exposure weakness back to serverless design, AWS service interaction, and secure engineering practice.

Chapter 4

Lesson 04: Insecure Cloud Configuration

4.1 Part 1: Goal and Vulnerability Summary

State the vulnerability type, main affected component, security impact, and the expected safe behavior for Insecure Cloud Configuration.

4.2 Part 2: Why This Works / Root Cause

Explain the security mistake that makes this cloud misconfiguration issue possible. Keep this conceptual and focused on root cause rather than symptoms.

4.3 Part 3: Environment and Setup

- DVSA Website URL / API endpoint: *Fill in the exact endpoint or page used.*
- AWS services involved: *For example API Gateway, Lambda, Cognito, S3, DynamoDB, SQS, IAM.*
- Relevant function, role, or log group: *Name the exact resources used in this lesson.*
- User or test context: *Attacker, victim, admin, or anonymous context as applicable.*
- Tools used: *Browser DevTools, curl, jq, AWS Console, AWS CLI, Policy Simulator, CloudWatch, etc.*

4.4 Part 4: Reproduction Steps

1. *Describe the baseline or normal behavior first.*
2. *List the exact steps that trigger the vulnerability.*

3. *Include the request, timing, role, or payload details needed for reproducibility.*
4. *Record the observable result that proves the issue.*

4.5 Part 5: Evidence and Proof

- *Attach the strongest pre-fix evidence for this lesson.*
- *Use only evidence that directly proves the claim.*
- *Redact real tokens, secrets, and temporary credentials before submission.*

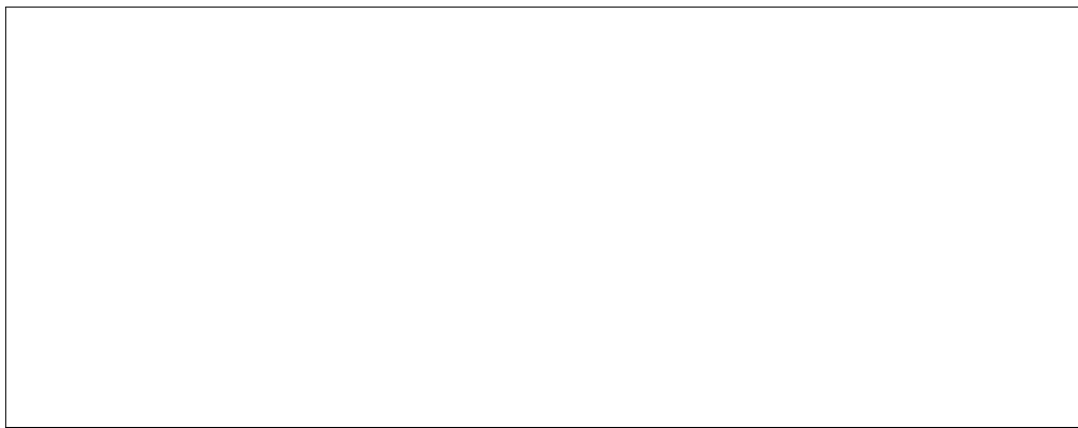


Figure 4.1: Replace with pre-fix evidence for Lesson 04 (Insecure Cloud Configuration).

4.6 Part 6: Fix Strategy / Probable Mitigation

- **Fix location:** *Identify the backend, IAM, cloud configuration, or workflow layer that must change.*
- **Security property restored:** *For example authentication integrity, authorization, least privilege, or safe input handling.*
- **Why this addresses root cause:** *Explain why the fix is real and not cosmetic.*

4.7 Part 7: Code / Config Changes

Paste the real code diff, policy change, CloudFormation change, or configuration update below.

Replace this box with the actual diff, code snippet, policy JSON, or configuration change.

Figure 4.2: Replace with the concrete code or configuration change for Lesson 04.

4.8 Part 8: Verification After Fix

1. *Repeat the same test that proved the vulnerability.*
2. *Show that the exploit no longer succeeds.*
3. *Also show that normal intended behavior still works.*

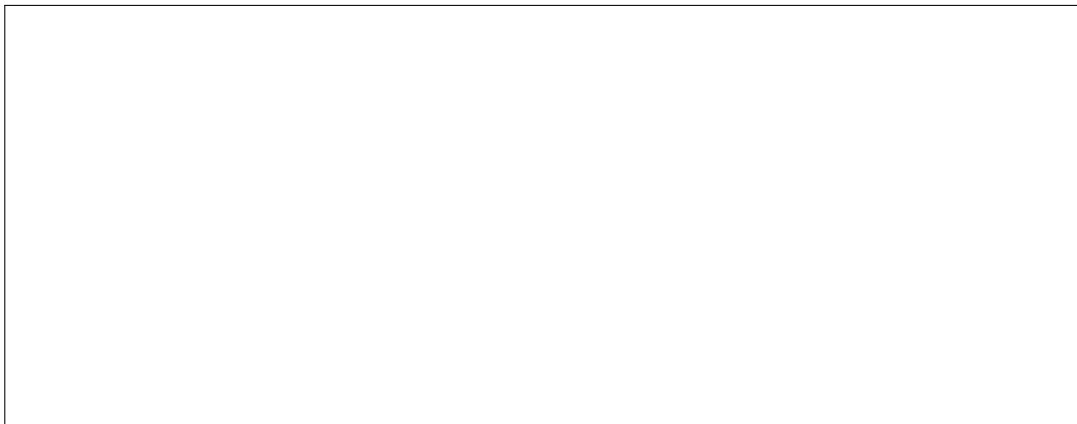


Figure 4.3: Replace with post-fix verification evidence for Lesson 04 (Insecure Cloud Configuration).

4.9 Part 9: Structured Operation and Security Analysis

Table A

Intended rule(s)	Artifacts used to infer the rule	Normal evidence	Exploit evidence
<i>State the intended security or workflow rule.</i>	<i>Identify the source used to infer the rule: UI flow, code path, IAM policy, logs, helper guide, etc.</i>	<i>Show normal behavior when the rule is respected.</i>	<i>Show behavior proving the rule was violated.</i>

Table B

Why the exploit is a deviation	Deviation class	Fix applied	Post-fix verification	Optional latency / timing note
<i>Explain why the observed result violates the intended rule.</i>	<i>Examples: authentication failure, access control failure, unsafe input handling, logic flaw, DoS, excessive privilege.</i>	<i>Summarize the real corrective action.</i>	<i>Describe the evidence that the deviation is closed.</i>	<i>Use this only when timing or latency matters.</i>

4.10 Part 10: Takeaway / Lessons Learned

End with a short security lesson that ties the cloud misconfiguration weakness back to serverless design, AWS service interaction, and secure engineering practice.

Chapter 5

Lesson 05: Broken Access Control

5.1 Part 1: Goal and Vulnerability Summary

State the vulnerability type, main affected component, security impact, and the expected safe behavior for Broken Access Control.

5.2 Part 2: Why This Works / Root Cause

Explain the security mistake that makes this authorization failure issue possible. Keep this conceptual and focused on root cause rather than symptoms.

5.3 Part 3: Environment and Setup

- DVSA Website URL / API endpoint: *Fill in the exact endpoint or page used.*
- AWS services involved: *For example API Gateway, Lambda, Cognito, S3, DynamoDB, SQS, IAM.*
- Relevant function, role, or log group: *Name the exact resources used in this lesson.*
- User or test context: *Attacker, victim, admin, or anonymous context as applicable.*
- Tools used: *Browser DevTools, curl, jq, AWS Console, AWS CLI, Policy Simulator, CloudWatch, etc.*

5.4 Part 4: Reproduction Steps

1. *Describe the baseline or normal behavior first.*
2. *List the exact steps that trigger the vulnerability.*
3. *Include the request, timing, role, or payload details needed for reproducibility.*

4. *Record the observable result that proves the issue.*

5.5 Part 5: Evidence and Proof

- *Attach the strongest pre-fix evidence for this lesson.*
- *Use only evidence that directly proves the claim.*
- *Redact real tokens, secrets, and temporary credentials before submission.*

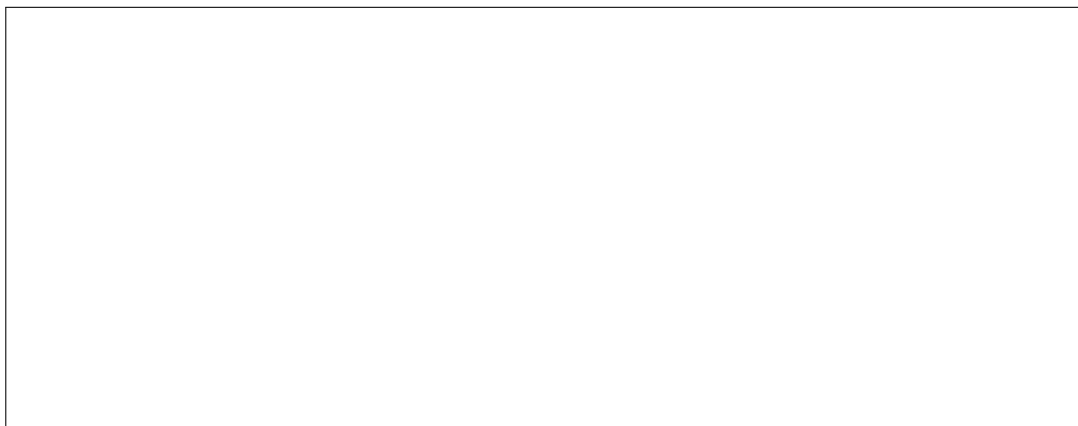


Figure 5.1: Replace with pre-fix evidence for Lesson 05 (Broken Access Control).

5.6 Part 6: Fix Strategy / Probable Mitigation

- **Fix location:** *Identify the backend, IAM, cloud configuration, or workflow layer that must change.*
- **Security property restored:** *For example authentication integrity, authorization, least privilege, or safe input handling.*
- **Why this addresses root cause:** *Explain why the fix is real and not cosmetic.*

5.7 Part 7: Code / Config Changes

Paste the real code diff, policy change, CloudFormation change, or configuration update below.

Replace this box with the actual diff, code snippet, policy JSON, or configuration change.

Figure 5.2: Replace with the concrete code or configuration change for Lesson 05.

5.8 Part 8: Verification After Fix

1. *Repeat the same test that proved the vulnerability.*
2. *Show that the exploit no longer succeeds.*
3. *Also show that normal intended behavior still works.*

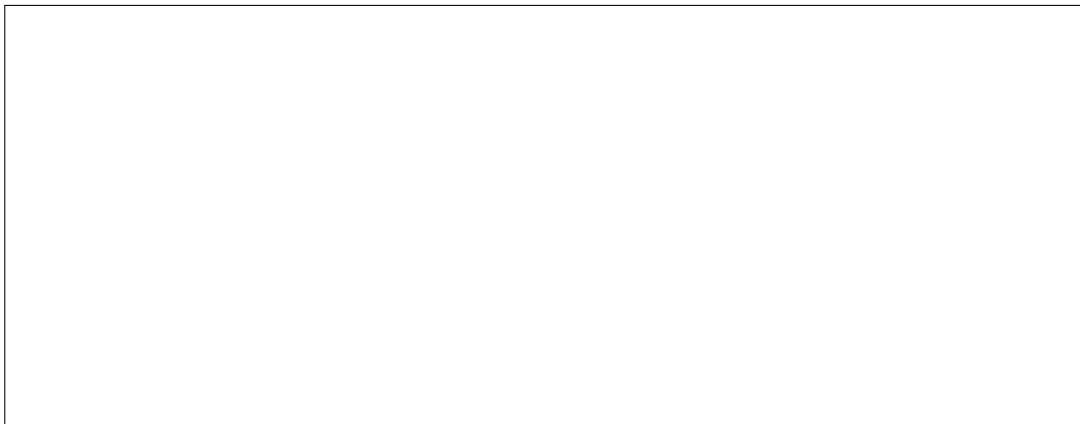


Figure 5.3: Replace with post-fix verification evidence for Lesson 05 (Broken Access Control).

5.9 Part 9: Structured Operation and Security Analysis

Table A

Intended rule(s)	Artifacts used to infer the rule	Normal evidence	Exploit evidence
<i>State the intended security or workflow rule.</i>	<i>Identify the source used to infer the rule: UI flow, code path, IAM policy, logs, helper guide, etc.</i>	<i>Show normal behavior when the rule is respected.</i>	<i>Show behavior proving the rule was violated.</i>

Table B

Why the exploit is a deviation	Deviation class	Fix applied	Post-fix verification	Optional latency / timing note
<i>Explain why the observed result violates the intended rule.</i>	<i>Examples: authentication failure, access control failure, unsafe input handling, logic flaw, DoS, excessive privilege.</i>	<i>Summarize the real corrective action.</i>	<i>Describe the evidence that the deviation is closed.</i>	<i>Use this only when timing or latency matters.</i>

5.10 Part 10: Takeaway / Lessons Learned

End with a short security lesson that ties the authorization failure weakness back to serverless design, AWS service interaction, and secure engineering practice.

Chapter 6

Lesson 06: Denial of Service

6.1 Part 1: Goal and Vulnerability Summary

State the vulnerability type, main affected component, security impact, and the expected safe behavior for Denial of Service.

6.2 Part 2: Why This Works / Root Cause

Explain the security mistake that makes this availability failure issue possible. Keep this conceptual and focused on root cause rather than symptoms.

6.3 Part 3: Environment and Setup

- DVSA Website URL / API endpoint: *Fill in the exact endpoint or page used.*
- AWS services involved: *For example API Gateway, Lambda, Cognito, S3, DynamoDB, SQS, IAM.*
- Relevant function, role, or log group: *Name the exact resources used in this lesson.*
- User or test context: *Attacker, victim, admin, or anonymous context as applicable.*
- Tools used: *Browser DevTools, curl, jq, AWS Console, AWS CLI, Policy Simulator, CloudWatch, etc.*

6.4 Part 4: Reproduction Steps

1. *Describe the baseline or normal behavior first.*
2. *List the exact steps that trigger the vulnerability.*
3. *Include the request, timing, role, or payload details needed for reproducibility.*

4. *Record the observable result that proves the issue.*

6.5 Part 5: Evidence and Proof

- *Attach the strongest pre-fix evidence for this lesson.*
- *Use only evidence that directly proves the claim.*
- *Redact real tokens, secrets, and temporary credentials before submission.*



Figure 6.1: Replace with pre-fix evidence for Lesson 06 (Denial of Service).

6.6 Part 6: Fix Strategy / Probable Mitigation

- **Fix location:** *Identify the backend, IAM, cloud configuration, or workflow layer that must change.*
- **Security property restored:** *For example authentication integrity, authorization, least privilege, or safe input handling.*
- **Why this addresses root cause:** *Explain why the fix is real and not cosmetic.*

6.7 Part 7: Code / Config Changes

Paste the real code diff, policy change, CloudFormation change, or configuration update below.

Replace this box with the actual diff, code snippet, policy JSON, or configuration change.

Figure 6.2: Replace with the concrete code or configuration change for Lesson 06.

6.8 Part 8: Verification After Fix

1. *Repeat the same test that proved the vulnerability.*
2. *Show that the exploit no longer succeeds.*
3. *Also show that normal intended behavior still works.*

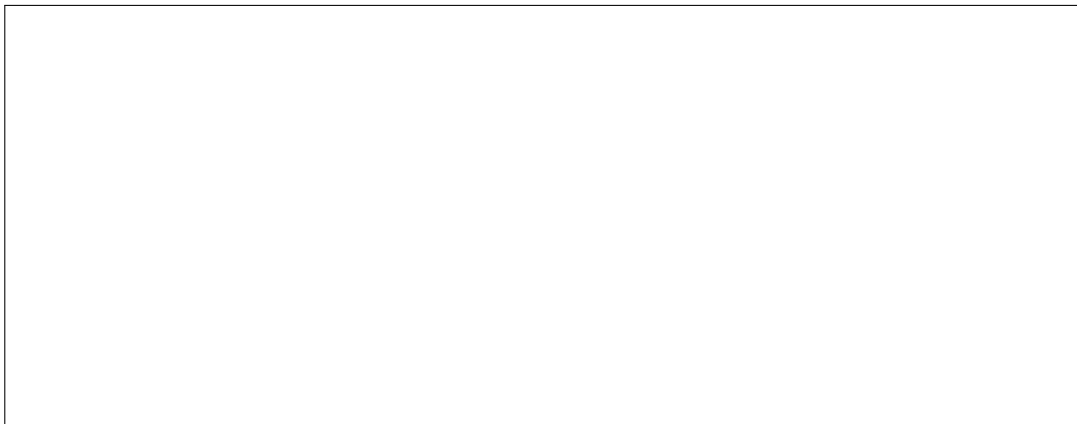


Figure 6.3: Replace with post-fix verification evidence for Lesson 06 (Denial of Service).

6.9 Part 9: Structured Operation and Security Analysis

Table A

Intended rule(s)	Artifacts used to infer the rule	Normal evidence	Exploit evidence
<i>State the intended security or workflow rule.</i>	<i>Identify the source used to infer the rule: UI flow, code path, IAM policy, logs, helper guide, etc.</i>	<i>Show normal behavior when the rule is respected.</i>	<i>Show behavior proving the rule was violated.</i>

Table B

Why the exploit is a deviation	Deviation class	Fix applied	Post-fix verification	Optional latency / timing note
<i>Explain why the observed result violates the intended rule.</i>	<i>Examples: authentication failure, access control failure, unsafe input handling, logic flaw, DoS, excessive privilege.</i>	<i>Summarize the real corrective action.</i>	<i>Describe the evidence that the deviation is closed.</i>	<i>Use this only when timing or latency matters.</i>

6.10 Part 10: Takeaway / Lessons Learned

End with a short security lesson that ties the availability failure weakness back to serverless design, AWS service interaction, and secure engineering practice.

Chapter 7

Lesson 07: Over-Privileged Function

7.1 Part 1: Goal and Vulnerability Summary

State the vulnerability type, main affected component, security impact, and the expected safe behavior for Over-Privileged Function.

7.2 Part 2: Why This Works / Root Cause

Explain the security mistake that makes this excessive IAM privilege issue possible. Keep this conceptual and focused on root cause rather than symptoms.

7.3 Part 3: Environment and Setup

- DVSA Website URL / API endpoint: *Fill in the exact endpoint or page used.*
- AWS services involved: *For example API Gateway, Lambda, Cognito, S3, DynamoDB, SQS, IAM.*
- Relevant function, role, or log group: *Name the exact resources used in this lesson.*
- User or test context: *Attacker, victim, admin, or anonymous context as applicable.*
- Tools used: *Browser DevTools, curl, jq, AWS Console, AWS CLI, Policy Simulator, CloudWatch, etc.*

7.4 Part 4: Reproduction Steps

1. *Describe the baseline or normal behavior first.*
2. *List the exact steps that trigger the vulnerability.*
3. *Include the request, timing, role, or payload details needed for reproducibility.*

4. *Record the observable result that proves the issue.*

7.5 Part 5: Evidence and Proof

- *Attach the strongest pre-fix evidence for this lesson.*
- *Use only evidence that directly proves the claim.*
- *Redact real tokens, secrets, and temporary credentials before submission.*



Figure 7.1: Replace with pre-fix evidence for Lesson 07 (Over-Privileged Function).

7.6 Part 6: Fix Strategy / Probable Mitigation

- **Fix location:** *Identify the backend, IAM, cloud configuration, or workflow layer that must change.*
- **Security property restored:** *For example authentication integrity, authorization, least privilege, or safe input handling.*
- **Why this addresses root cause:** *Explain why the fix is real and not cosmetic.*

7.7 Part 7: Code / Config Changes

Paste the real code diff, policy change, CloudFormation change, or configuration update below.

Replace this box with the actual diff, code snippet, policy JSON, or configuration change.

Figure 7.2: Replace with the concrete code or configuration change for Lesson 07.

7.8 Part 8: Verification After Fix

1. *Repeat the same test that proved the vulnerability.*
2. *Show that the exploit no longer succeeds.*
3. *Also show that normal intended behavior still works.*

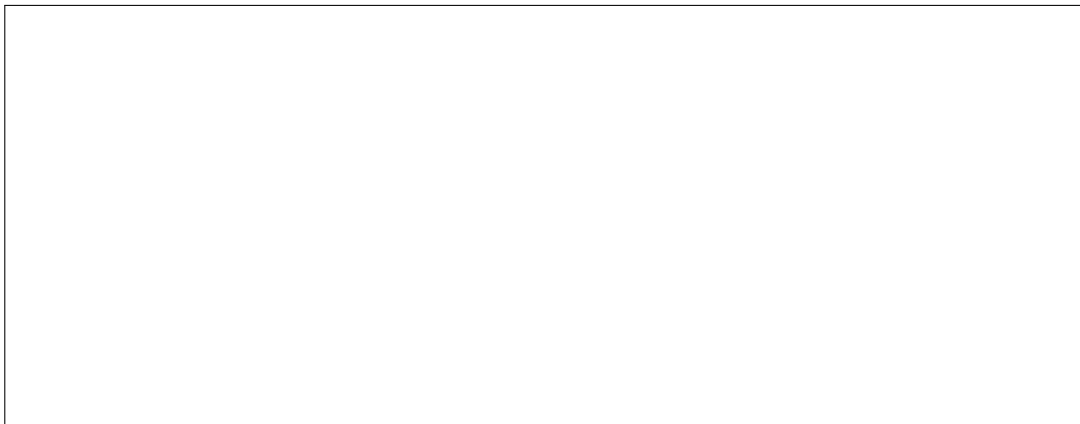


Figure 7.3: Replace with post-fix verification evidence for Lesson 07 (Over-Privileged Function).

7.9 Part 9: Structured Operation and Security Analysis

Table A

Intended rule(s)	Artifacts used to infer the rule	Normal evidence	Exploit evidence
<i>State the intended security or workflow rule.</i>	<i>Identify the source used to infer the rule: UI flow, code path, IAM policy, logs, helper guide, etc.</i>	<i>Show normal behavior when the rule is respected.</i>	<i>Show behavior proving the rule was violated.</i>

Table B

Why the exploit is a deviation	Deviation class	Fix applied	Post-fix verification	Optional latency / timing note
<i>Explain why the observed result violates the intended rule.</i>	<i>Examples: authentication failure, access control failure, unsafe input handling, logic flaw, DoS, excessive privilege.</i>	<i>Summarize the real corrective action.</i>	<i>Describe the evidence that the deviation is closed.</i>	<i>Use this only when timing or latency matters.</i>

7.10 Part 10: Takeaway / Lessons Learned

End with a short security lesson that ties the excessive IAM privilege weakness back to serverless design, AWS service interaction, and secure engineering practice.

Chapter 8

Lesson 08: Logic Vulnerabilities

8.1 Part 1: Goal and Vulnerability Summary

State the vulnerability type, main affected component, security impact, and the expected safe behavior for Logic Vulnerabilities.

8.2 Part 2: Why This Works / Root Cause

Explain the security mistake that makes this workflow or race-condition failure issue possible. Keep this conceptual and focused on root cause rather than symptoms.

8.3 Part 3: Environment and Setup

- DVSA Website URL / API endpoint: *Fill in the exact endpoint or page used.*
- AWS services involved: *For example API Gateway, Lambda, Cognito, S3, DynamoDB, SQS, IAM.*
- Relevant function, role, or log group: *Name the exact resources used in this lesson.*
- User or test context: *Attacker, victim, admin, or anonymous context as applicable.*
- Tools used: *Browser DevTools, curl, jq, AWS Console, AWS CLI, Policy Simulator, CloudWatch, etc.*

8.4 Part 4: Reproduction Steps

1. *Describe the baseline or normal behavior first.*
2. *List the exact steps that trigger the vulnerability.*
3. *Include the request, timing, role, or payload details needed for reproducibility.*

4. *Record the observable result that proves the issue.*

8.5 Part 5: Evidence and Proof

- *Attach the strongest pre-fix evidence for this lesson.*
- *Use only evidence that directly proves the claim.*
- *Redact real tokens, secrets, and temporary credentials before submission.*



Figure 8.1: Replace with pre-fix evidence for Lesson 08 (Logic Vulnerabilities).

8.6 Part 6: Fix Strategy / Probable Mitigation

- **Fix location:** *Identify the backend, IAM, cloud configuration, or workflow layer that must change.*
- **Security property restored:** *For example authentication integrity, authorization, least privilege, or safe input handling.*
- **Why this addresses root cause:** *Explain why the fix is real and not cosmetic.*

8.7 Part 7: Code / Config Changes

Paste the real code diff, policy change, CloudFormation change, or configuration update below.

Replace this box with the actual diff, code snippet, policy JSON, or configuration change.

Figure 8.2: Replace with the concrete code or configuration change for Lesson 08.

8.8 Part 8: Verification After Fix

1. *Repeat the same test that proved the vulnerability.*
2. *Show that the exploit no longer succeeds.*
3. *Also show that normal intended behavior still works.*

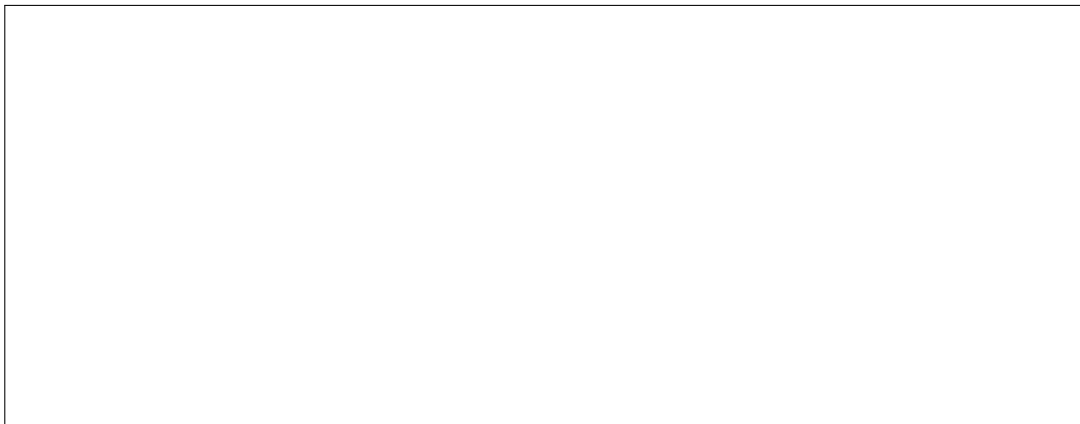


Figure 8.3: Replace with post-fix verification evidence for Lesson 08 (Logic Vulnerabilities).

8.9 Part 9: Structured Operation and Security Analysis

Table A

Intended rule(s)	Artifacts used to infer the rule	Normal evidence	Exploit evidence
<i>State the intended security or workflow rule.</i>	<i>Identify the source used to infer the rule: UI flow, code path, IAM policy, logs, helper guide, etc.</i>	<i>Show normal behavior when the rule is respected.</i>	<i>Show behavior proving the rule was violated.</i>

Table B

Why the exploit is a deviation	Deviation class	Fix applied	Post-fix verification	Optional latency / timing note
<i>Explain why the observed result violates the intended rule.</i>	<i>Examples: authentication failure, access control failure, unsafe input handling, logic flaw, DoS, excessive privilege.</i>	<i>Summarize the real corrective action.</i>	<i>Describe the evidence that the deviation is closed.</i>	<i>Use this only when timing or latency matters.</i>

8.10 Part 10: Takeaway / Lessons Learned

End with a short security lesson that ties the workflow or race-condition failure weakness back to serverless design, AWS service interaction, and secure engineering practice.

Chapter 9

Lesson 09: Vulnerable Dependencies

9.1 Part 1: Goal and Vulnerability Summary

State the vulnerability type, main affected component, security impact, and the expected safe behavior for Vulnerable Dependencies.

9.2 Part 2: Why This Works / Root Cause

Explain the security mistake that makes this dependency risk issue possible. Keep this conceptual and focused on root cause rather than symptoms.

9.3 Part 3: Environment and Setup

- DVSA Website URL / API endpoint: *Fill in the exact endpoint or page used.*
- AWS services involved: *For example API Gateway, Lambda, Cognito, S3, DynamoDB, SQS, IAM.*
- Relevant function, role, or log group: *Name the exact resources used in this lesson.*
- User or test context: *Attacker, victim, admin, or anonymous context as applicable.*
- Tools used: *Browser DevTools, curl, jq, AWS Console, AWS CLI, Policy Simulator, CloudWatch, etc.*

9.4 Part 4: Reproduction Steps

1. *Describe the baseline or normal behavior first.*
2. *List the exact steps that trigger the vulnerability.*
3. *Include the request, timing, role, or payload details needed for reproducibility.*

4. *Record the observable result that proves the issue.*

9.5 Part 5: Evidence and Proof

- *Attach the strongest pre-fix evidence for this lesson.*
- *Use only evidence that directly proves the claim.*
- *Redact real tokens, secrets, and temporary credentials before submission.*

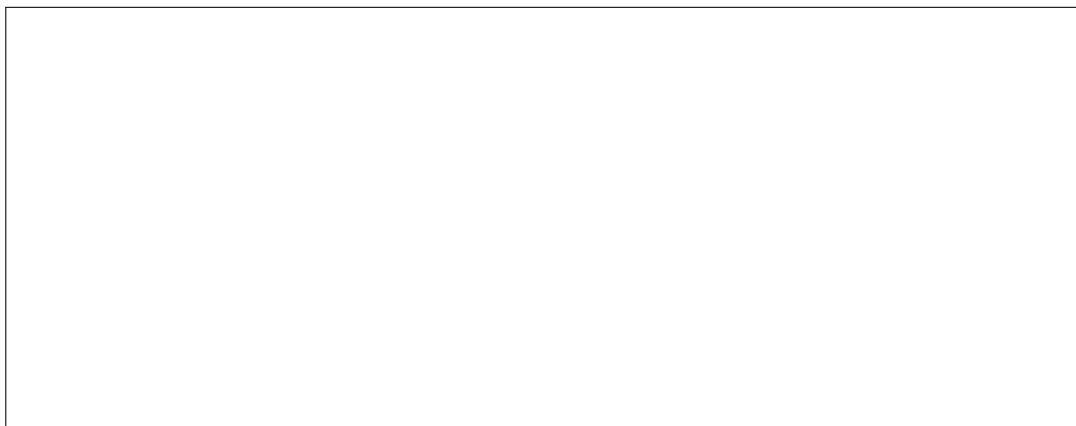


Figure 9.1: Replace with pre-fix evidence for Lesson 09 (Vulnerable Dependencies).

9.6 Part 6: Fix Strategy / Probable Mitigation

- **Fix location:** *Identify the backend, IAM, cloud configuration, or workflow layer that must change.*
- **Security property restored:** *For example authentication integrity, authorization, least privilege, or safe input handling.*
- **Why this addresses root cause:** *Explain why the fix is real and not cosmetic.*

9.7 Part 7: Code / Config Changes

Paste the real code diff, policy change, CloudFormation change, or configuration update below.

Replace this box with the actual diff, code snippet, policy JSON, or configuration change.

Figure 9.2: Replace with the concrete code or configuration change for Lesson 09.

9.8 Part 8: Verification After Fix

1. *Repeat the same test that proved the vulnerability.*
2. *Show that the exploit no longer succeeds.*
3. *Also show that normal intended behavior still works.*

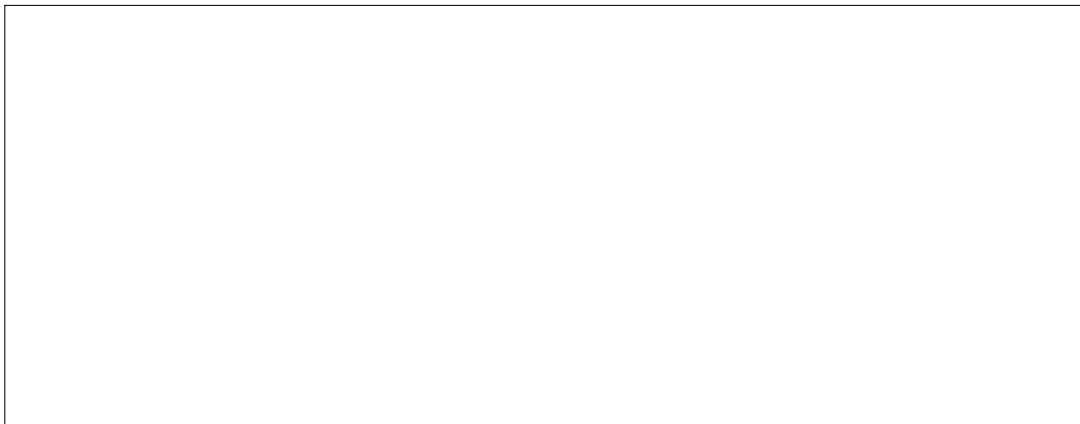


Figure 9.3: Replace with post-fix verification evidence for Lesson 09 (Vulnerable Dependencies).

9.9 Part 9: Structured Operation and Security Analysis

Table A

Intended rule(s)	Artifacts used to infer the rule	Normal evidence	Exploit evidence
<i>State the intended security or workflow rule.</i>	<i>Identify the source used to infer the rule: UI flow, code path, IAM policy, logs, helper guide, etc.</i>	<i>Show normal behavior when the rule is respected.</i>	<i>Show behavior proving the rule was violated.</i>

Table B

Why the exploit is a deviation	Deviation class	Fix applied	Post-fix verification	Optional latency / timing note
<i>Explain why the observed result violates the intended rule.</i>	<i>Examples: authentication failure, access control failure, unsafe input handling, logic flaw, DoS, excessive privilege.</i>	<i>Summarize the real corrective action.</i>	<i>Describe the evidence that the deviation is closed.</i>	<i>Use this only when timing or latency matters.</i>

9.10 Part 10: Takeaway / Lessons Learned

End with a short security lesson that ties the dependency risk weakness back to serverless design, AWS service interaction, and secure engineering practice.

Chapter 10

Lesson 10: Unhandled Exceptions

10.1 Part 1: Goal and Vulnerability Summary

State the vulnerability type, main affected component, security impact, and the expected safe behavior for Unhandled Exceptions.

10.2 Part 2: Why This Works / Root Cause

Explain the security mistake that makes this unsafe error handling issue possible. Keep this conceptual and focused on root cause rather than symptoms.

10.3 Part 3: Environment and Setup

- DVSA Website URL / API endpoint: *Fill in the exact endpoint or page used.*
- AWS services involved: *For example API Gateway, Lambda, Cognito, S3, DynamoDB, SQS, IAM.*
- Relevant function, role, or log group: *Name the exact resources used in this lesson.*
- User or test context: *Attacker, victim, admin, or anonymous context as applicable.*
- Tools used: *Browser DevTools, curl, jq, AWS Console, AWS CLI, Policy Simulator, CloudWatch, etc.*

10.4 Part 4: Reproduction Steps

1. *Describe the baseline or normal behavior first.*
2. *List the exact steps that trigger the vulnerability.*
3. *Include the request, timing, role, or payload details needed for reproducibility.*

4. *Record the observable result that proves the issue.*

10.5 Part 5: Evidence and Proof

- *Attach the strongest pre-fix evidence for this lesson.*
- *Use only evidence that directly proves the claim.*
- *Redact real tokens, secrets, and temporary credentials before submission.*

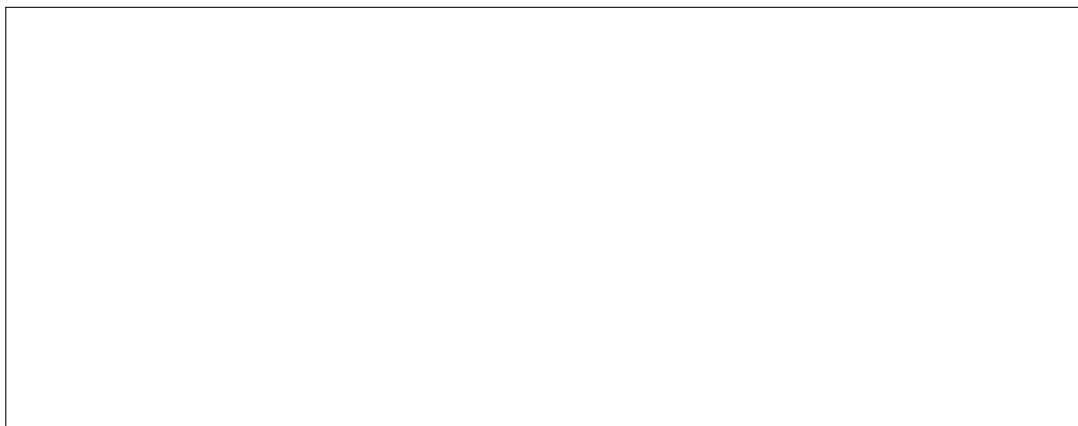


Figure 10.1: Replace with pre-fix evidence for Lesson 10 (Unhandled Exceptions).

10.6 Part 6: Fix Strategy / Probable Mitigation

- **Fix location:** *Identify the backend, IAM, cloud configuration, or workflow layer that must change.*
- **Security property restored:** *For example authentication integrity, authorization, least privilege, or safe input handling.*
- **Why this addresses root cause:** *Explain why the fix is real and not cosmetic.*

10.7 Part 7: Code / Config Changes

Paste the real code diff, policy change, CloudFormation change, or configuration update below.

Replace this box with the actual diff, code snippet, policy JSON, or configuration change.

Figure 10.2: Replace with the concrete code or configuration change for Lesson 10.

10.8 Part 8: Verification After Fix

1. *Repeat the same test that proved the vulnerability.*
2. *Show that the exploit no longer succeeds.*
3. *Also show that normal intended behavior still works.*

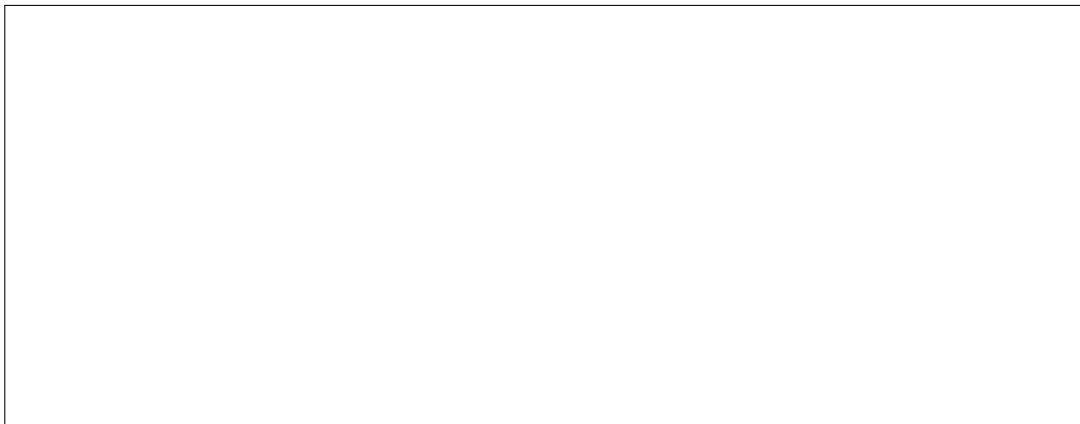


Figure 10.3: Replace with post-fix verification evidence for Lesson 10 (Unhandled Exceptions).

10.9 Part 9: Structured Operation and Security Analysis

Table A

Intended rule(s)	Artifacts used to infer the rule	Normal evidence	Exploit evidence
<i>State the intended security or workflow rule.</i>	<i>Identify the source used to infer the rule: UI flow, code path, IAM policy, logs, helper guide, etc.</i>	<i>Show normal behavior when the rule is respected.</i>	<i>Show behavior proving the rule was violated.</i>

Table B

Why the exploit is a deviation	Deviation class	Fix applied	Post-fix verification	Optional latency / timing note
<i>Explain why the observed result violates the intended rule.</i>	<i>Examples: authentication failure, access control failure, unsafe input handling, logic flaw, DoS, excessive privilege.</i>	<i>Summarize the real corrective action.</i>	<i>Describe the evidence that the deviation is closed.</i>	<i>Use this only when timing or latency matters.</i>

10.10 Part 10: Takeaway / Lessons Learned

End with a short security lesson that ties the unsafe error handling weakness back to serverless design, AWS service interaction, and secure engineering practice.

Optional Bonus Findings

*Use this section only if you discover valid additional findings beyond the 10 official lessons.
Repeat the same reporting discipline if you include bonus work.*

Conclusion

Summarize the project outcomes, the most important remediation themes, and the main security lessons learned from DVSA.